

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ
GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

PEDRO DA FONSECA SCHWERTNER

SOPHIA OELKE VERDI

LÍBIA CANICEIRO DA SILVEIRA

MARIA PAULA DE JESUS SANTOS

RELATÓRIO PROJETO DE EXTENSÃO - SÍNDROME DO X FRÁGIL

Curitiba, Paraná

2025

PEDRO DA FONSECA SCHWERTNER

SOPHIA OELKE VERDI

LÍBIA CANICEIRO DA SILVEIRA

MARIA PAULA DE JESUS SANTOS

RELATÓRIO PROJETO DE EXTENSÃO - SÍNDROME DO X FRÁGIL

Trabalho discente efetivo
apresentado à Pontifícia
Universidade Católica do
Paraná - PUCPR.

Orientador: Prof. Andrey
Cabral Meira

Curitiba, Paraná

2025

RESUMO

O presente relatório descreve o desenvolvimento de um sistema de informação voltado à triagem populacional da Síndrome do X Frágil (SXF), realizado em parceria com o Instituto Buko Kaesemodel (IBK). A SXF é uma condição genética de alto impacto no desenvolvimento intelectual, cujo diagnóstico molecular definitivo possui elevado custo. O objetivo deste projeto foi automatizar uma lista de verificação clínica validada para identificar pacientes com alta probabilidade de portarem a síndrome, otimizando o encaminhamento para exames genéticos. A metodologia envolveu o desenvolvimento de uma aplicação Full Stack utilizando React.js no frontend, Python com FastAPI no backend, e banco de dados MySQL via Docker. O sistema implementa um algoritmo de score ponderado diferenciado por sexo biológico e define limiares de decisão específicos (0,56 para homens e 0,55 para mulheres). Como resultado, obteve-se uma plataforma segura, responsiva e de baixo custo, capaz de gerenciar perfis de acesso distintos (administradores, médicos e pacientes) e fornecer recomendações automatizadas de encaminhamento clínico. Conclui-se que a ferramenta digital potencializa o diagnóstico precoce e a gestão de dados epidemiológicos da SXF no Brasil.

Palavras-chave: Síndrome do X Frágil. Triagem Clínica. Engenharia de Software. Score Ponderado. Saúde Digital.

ABSTRACT

This report describes the development of an information system for the population screening of Fragile X Syndrome (FXS), carried out in partnership with the Buko Kaesemodel Institute (IBK). FXS is a genetic condition with a high impact on intellectual development, and its definitive molecular diagnosis is costly. The objective of this project was to automate a validated clinical checklist to identify patients with a high probability of having the syndrome, optimizing the referral for genetic testing. The methodology involved the development of a Full Stack application using React.js for the frontend, Python with FastAPI for the backend, and a MySQL database via Docker. The system implements a weighted score algorithm differentiated by biological sex and defines specific decision thresholds (0.56 for males and 0.55 for females). As a result, a secure, responsive, and low-cost platform was obtained, capable of managing distinct access profiles (administrators, doctors, and patients) and providing automated clinical referral recommendations. It is concluded that the digital tool enhances early diagnosis and the management of FXS epidemiological data in Brazil.

Keywords: Fragile X Syndrome. Clinical Screening. Software Engineering. Weighted Score. Digital Health.

SUMÁRIO

RESUMO.....	3
ABSTRACT.....	4
SUMÁRIO.....	5
1. INTRODUÇÃO.....	6
2. DESENVOLVIMENTO.....	7
2.1. Descrição do Cenário (Minimundo).....	7
2.2. Tecnologias.....	8
2.2.1. Frontend.....	8
2.2.1.1. Framework: React.js.....	8
2.2.1.2. Build Tool: Vite.....	8
2.2.1.3. Bibliotecas e Ferramentas.....	9
2.2.1.4. Design de Telas: Figma.....	9
2.2.1.4.1. Telas originais do Figma:.....	10
2.2.1.4.2. Telas finais da aplicação:.....	12
2.2.2. Backend.....	14
2.2.2.1. Runtime: Python / FastAPI.....	14
2.2.2.2. ORM: SQLAlchemy.....	14
2.2.2.3. Bibliotecas Auxiliares.....	14
2.2.3. Database.....	15
2.2.4. Requisitos do Sistema.....	15
2.2.4.1. Requisitos Funcionais (RF).....	16
2.2.4.2. Requisitos Não Funcionais (RNF).....	17
2.3. Deployment (Implantação).....	18
3. CONCLUSÃO.....	19
REFERÊNCIAS.....	20

1. INTRODUÇÃO

A Síndrome do X Frágil (SXF) é uma condição genética de herança ligada ao cromossomo X, sendo a causa mais comum de deficiência intelectual hereditária e a principal causa genética conhecida para o Transtorno do Espectro Autista (TEA). Apesar de sua relevância clínica, o diagnóstico definitivo no Brasil enfrenta barreiras significativas devido ao alto custo e à baixa disponibilidade de testes moleculares, como o PCR e o Southern Blot. Diante desse cenário, a triagem populacional baseada em indicadores clínicos apresenta-se como uma estratégia vital para a otimização de recursos e o diagnóstico precoce.

Este projeto detalha o desenvolvimento de um sistema de informação web destinado ao Instituto Buko Kaesemodel (IBK). A solução visa automatizar a aplicação de uma lista de verificação (checklist) validada, composta por 12 indicadores clínicos. Através de um algoritmo de score ponderado, que diferencia pesos conforme o sexo biológico do paciente, o sistema busca fornecer uma ferramenta de baixo custo e alta precisão para auxiliar profissionais de saúde na identificação de casos suspeitos. A implementação utiliza tecnologias modernas de desenvolvimento full stack, garantindo segurança, escalabilidade e acessibilidade para médicos, administradores e pacientes.

2. DESENVOLVIMENTO

2.1. Descrição do Cenário (Minimundo)

O Instituto Buko Kaesemodel (IBK) deseja um sistema de informação para automatizar o desenvolvimento e validação de uma lista de verificação clínica para triagem populacional da Síndrome do X Frágil (SXF) no Brasil. O objetivo principal é fornecer uma ferramenta de baixo custo que auxilie profissionais de saúde a identificar padrões clínicos e direcionar casos de alta probabilidade para testes genéticos confirmatórios (PCR e Southern Blot), que possuem custo elevado e acesso limitado.

O sistema deve gerenciar o cadastro de pacientes, registrando informações como nome, idade, sexo biológico e o responsável legal. A triagem consiste na aplicação de um checklist clínico composto por 12 indicadores (sinais e sintomas), que são: deficiência intelectual, face alongada/orelhas de abano, macroorquidismo, hipermobilidade articular, dificuldades de aprendizagem, déficit de atenção, movimentos repetitivos, atraso na fala, hiperatividade, evitar contato visual, evitar contato físico e agressividade. Para cada avaliação, o sistema deve calcular um score final baseado em pesos específicos atribuídos a cada sintoma. Esses pesos variam obrigatoriamente de acordo com o sexo do paciente. Por exemplo, a “Deficiência Intelectual” possui peso 0,32 para homens e 0,20 para mulheres, enquanto “Dificuldades de Aprendizagem” possui peso 0,18 para homens e 0,28 para mulheres. O sistema deve comparar o score total calculado com limiares de decisão (pontuação de corte). Os limiares validados são de 0,56 para o sexo masculino e 0,55 para o sexo feminino. Caso a pontuação seja igual ou superior ao limiar, o sistema deve gerar automaticamente uma recomendação de encaminhamento para o teste genético FMR1.

O acesso ao sistema é restrito. Usuário do tipo “Administrador” pode gerenciar o cadastro de outros usuários e visualizar relatórios consolidados de todas

as avaliações por data e filtros diversos. Usuários do tipo “Médico” podem apenas realizar o cadastro de seus pacientes, preencher os formulários de triagem e consultar os históricos e relatórios das avaliações realizadas por eles próprios. Usuários do tipo “Paciente” podem apenas consultar seus respectivos cadastros, suas avaliações e informações sobre o exame, se precisam ou não fazer. O sistema deve permitir a impressão dos resultados para os pacientes.

2.2. Tecnologias

O projeto adota uma arquitetura Full Stack moderna, organizada em um repositório único (Monorepo) no GitHub, facilitando a gestão do código, o versionamento sincronizado entre as partes do sistema e a integração contínua (CI/CD).

2.2.1. Frontend

O frontend foi projetado para oferecer uma interface de alta performance, priorizando a agilidade necessária em ambientes ambulatoriais.

2.2.1.1. Framework: [React.js](https://react.dev/)

A escolha do React.js baseia-se na sua arquitetura baseada em componentes reutilizáveis, o que permite manter a consistência visual em todo o sistema. Sua principal vantagem para este projeto é o uso do Virtual DOM, que permite atualizar em tempo real o cálculo do score de triagem conforme o médico marca os sintomas, sem a necessidade de recarregar a página.

2.2.1.2. Build Tool: Vite

Diferente de ferramentas tradicionais, o Vite utiliza o poder dos Native ESM para fornecer um ambiente de desenvolvimento extremamente rápido. No processo de build, ele otimiza os ativos (assets) e realiza o tree-shaking (remoção de código não utilizado), garantindo que o arquivo final seja leve e carregue rapidamente, mesmo em conexões de internet instáveis.

2.2.1.3. Bibliotecas e Ferramentas

React DOM: Responsável pela renderização dos componentes React na árvore do DOM do navegador, sendo a camada fundamental que conecta a lógica dos componentes à interface visual exibida ao usuário.

React Router DOM: Utilizado para o gerenciamento de rotas e navegação no lado do cliente. Permite a criação de uma experiência de Single Page Application (SPA), onde a transição entre as páginas dos diferentes perfis (Administrador, Médico, Paciente) ocorre sem recarregamento completo da página.

jsPDF: Biblioteca JavaScript utilizada para a geração de documentos PDF diretamente no navegador. No sistema, é empregada para produzir o relatório imprimível das consultas de triagem, permitindo que médicos e pacientes exportem os resultados das avaliações.

2.2.1.4. Design de Telas: Figma

O Figma foi adotado como a ferramenta central de Design de Interface (UI) e Experiência do Usuário (UX), atuando como o elo entre a concepção teórica do checklist clínico e a implementação técnica no Frontend. A utilização do Figma permitiu a criação de um Protótipo de Alta Fidelidade, simulando com precisão a interatividade final do sistema antes do início da escrita de qualquer linha de código.

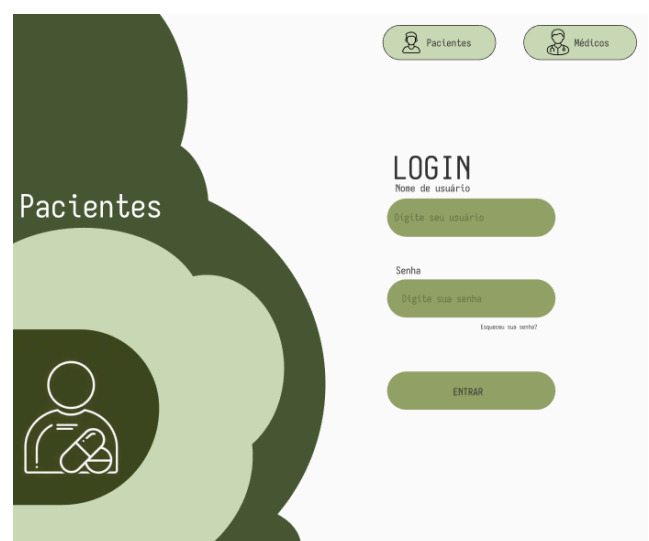
Os principais pontos de desenvolvimento nesta etapa incluíram:

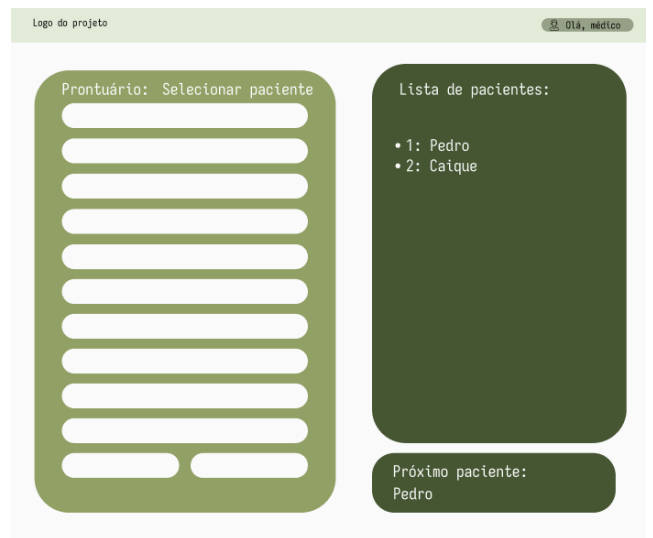
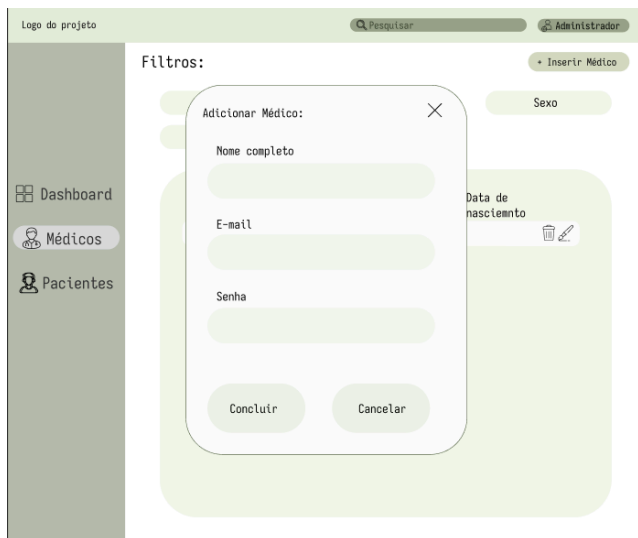
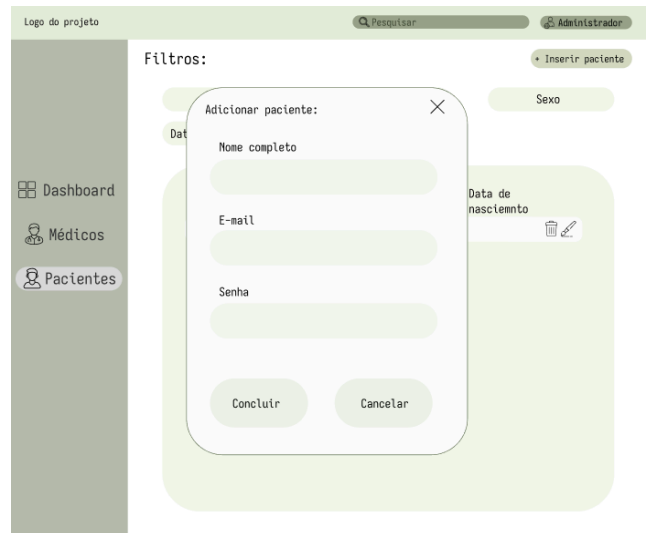
- **Arquitetura de Informação e Fluxo de Navegação:** Foi desenhada a jornada completa dos três perfis de usuário (Administrador, Médico e Paciente). Para o perfil médico, o foco foi a redução da carga cognitiva, organizando os 12 indicadores da Síndrome do X Frágil em uma sequência lógica que facilita o preenchimento rápido durante o atendimento clínico.
- **Acessibilidade e Usabilidade Clínica:** O design priorizou o contraste de cores e o tamanho das fontes para garantir a legibilidade em ambientes de saúde (muitas vezes com iluminação variada).

Telas desenvolvidas:

- Página de Login geral
- Tela inicial do sistema
- Página do administrador (Gerenciamento de contas)
- Página de consulta dos médicos para os pacientes atendidos
- Página de erro 404
- Página de recuperação de senha

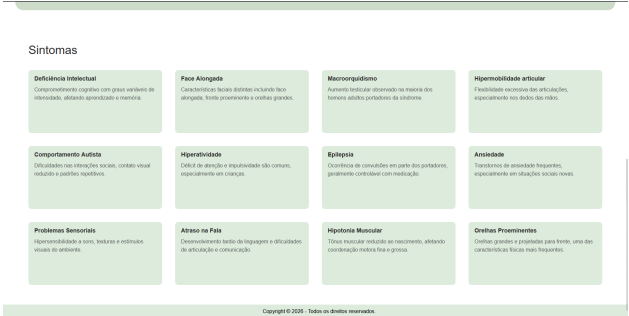
2.2.1.4.1. Telas originais do Figma:



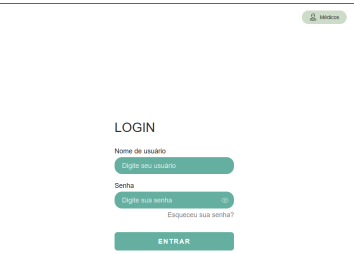
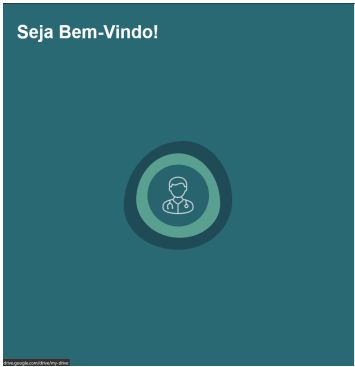


2.2.1.4.2. Telas finais da aplicação:

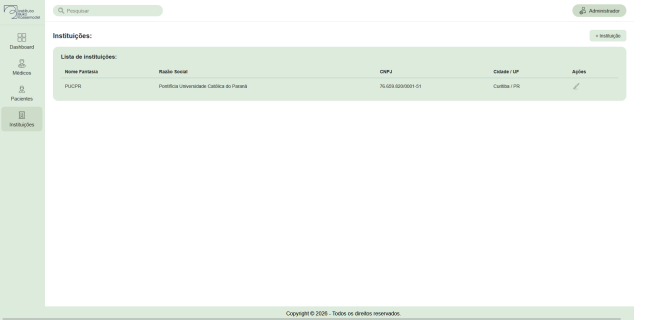
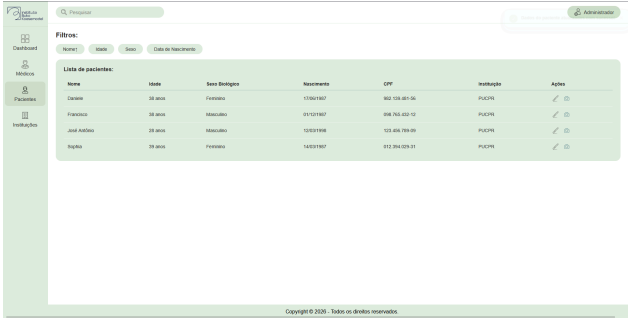
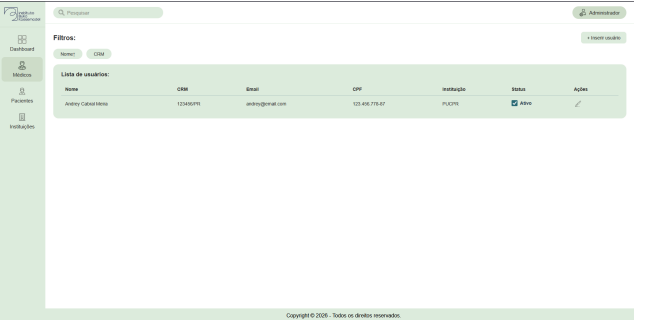
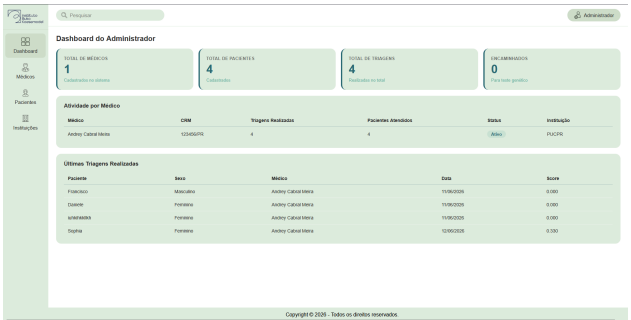
Tela inicial:



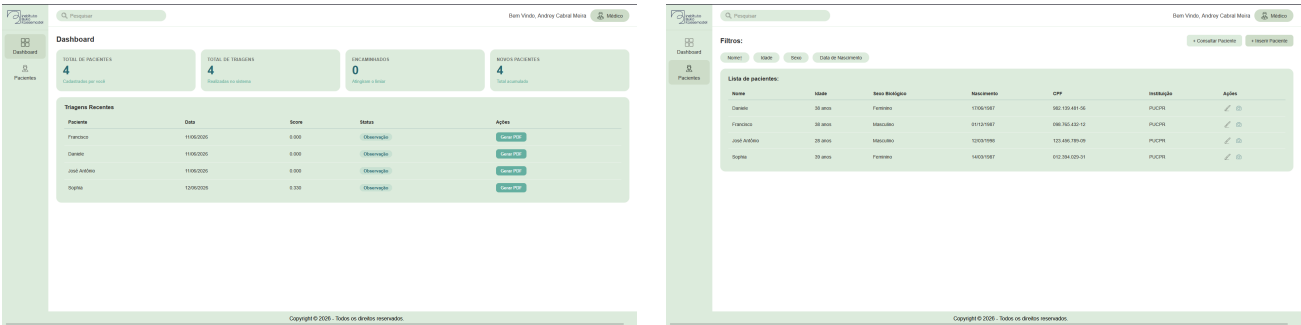
Tela de Login:



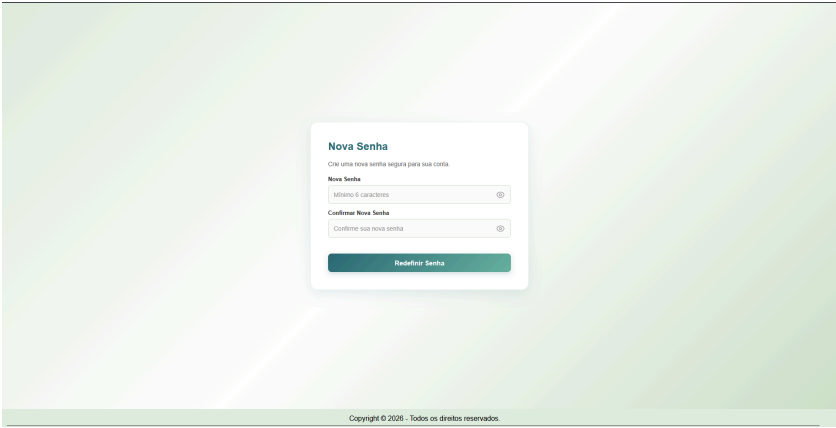
Telas do administrador:



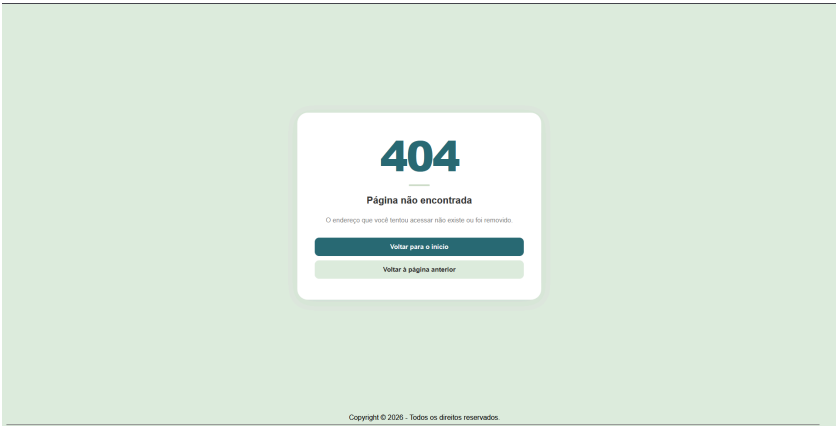
Telas do médico:



Tela de recuperação de senha:



Tela de página não encontrada (404):



2.2.2. Backend

O backend foi estruturado para ser uma API REST robusta, focada na integridade dos cálculos e na segurança dos dados sensíveis dos pacientes.

2.2.2.1. Runtime: Python / FastAPI

O Python foi escolhido como linguagem do backend pela sua maturidade no ecossistema científico e pela clareza do código, especialmente em operações matemáticas como o cálculo do score ponderado. O framework FastAPI, construído sobre o padrão ASGI com Starlette como base, oferece alta performance e validação automática de dados via type hints nativos do Python. A integração com Mangum permite que a aplicação seja executada em ambientes serverless, além de facilitar o deploy local via Docker.

2.2.2.2. ORM: SQLAlchemy

O SQLAlchemy atua como a camada de abstração entre o código Python e o banco de dados MySQL. Por meio de modelos declarativos, ele mapeia as entidades do sistema (pacientes, avaliações, usuários) para tabelas relacionais, garantindo que as operações de consulta e persistência sejam tipadas e seguras. As migrações de esquema são gerenciadas de forma versionada, assegurando consistência entre os ambientes de desenvolvimento e produção.

2.2.2.3. Bibliotecas Auxiliares

O backend conta com um conjunto de bibliotecas especializadas para suprir as demandas funcionais do sistema:

- MIME e smtplib: Responsáveis pelo envio de notificações por e-mail aos pacientes. O smtplib gerencia a conexão com o servidor de e-mail, enquanto o MIME estrutura o conteúdo das mensagens com formatação adequada.
- hashlib: Utilizado para a criptografia de senhas com o algoritmo SHA-256, garantindo que as credenciais dos usuários sejam armazenadas de forma segura no banco de dados, em conformidade com boas práticas de segurança.

- ReportLab: Biblioteca Python empregada na geração de documentos PDF no servidor. É utilizada para produzir os relatórios formais das consultas de triagem, que podem ser impressos ou baixados pelo médico.
- datetime: Utilizada para o registro e manipulação de datas e horários das avaliações, garantindo o correto histórico temporal dos atendimentos.
- unicodedata: Empregada no tratamento e normalização de strings com caracteres especiais, essencial para o processamento correto de nomes e dados em português.
- Starlette: Framework ASGI que serve de base para o FastAPI, fornecendo os componentes fundamentais de rápida execução assíncrona, middlewares e gerenciamento de requisições HTTP.
- Mangum: Adaptador que permite a execução da aplicação FastAPI/ASGI em ambientes serverless, além de facilitar a integração com containers Docker no ambiente local.

2.2.3. Database

Sistema: MySQL 8.0

Como os dados de saúde possuem uma estrutura clara e relacional (um paciente possui muitas avaliações, um médico atende muitos pacientes), o MySQL foi a escolha natural. Ele oferece suporte a transações ACID, garantindo que o registro de uma triagem e o cálculo do seu score sejam gravados de forma atômica e segura, evitando perda de dados.

Ambiente: Docker

A containerização com Docker isola o banco de dados das dependências do sistema operacional do desenvolvedor. Através de um arquivo docker-compose.yml, é possível subir uma instância do MySQL com todas as configurações de portas, volumes (para persistência de dados) e variáveis de ambiente pré-definidas, garantindo que o sistema funcione exatamente da mesma forma no computador de cada integrante do grupo.

2.2.4. Requisitos do Sistema

2.2.4.1. Requisitos Funcionais (RF)

Identificador	Descrição	Prioridade	Depende de
RF01	O sistema deve permitir que Profissional de Saúde cadastre o paciente com dados como nome, sexo, idade, e dados do responsável.	Alta	RF07
RF02	O sistema deve conter um Formulário Clínico Interativo para a inserção dos 12 sintomas a fim de que o Profissional de Saúde possa avaliar o paciente e inserir os dados de forma binária (presente/ ausente).	Alta	RF07, RF01
RF03	O sistema deve calcular um score final automaticamente baseado em pesos específicos atribuídos a cada sintoma de acordo com o sexo do paciente.	Alta	RF01, RF02
RF04	O sistema deve comparar o score total calculado com limiares de decisão (pontuação de corte).	Alta	RF01, RF02, RF03
RF05	O sistema deve gerar automaticamente uma recomendação de encaminhamento para o teste genético FMR1, caso o limiar de risco seja atingido.	Alta	RF04
RF06	O sistema deve armazenar o histórico de Avaliações e Relatórios e permitir que Profissionais de Saúde possam realizar consultas futuras.	Alta	RF01, RF02, RF03, RF04
RF07	O sistema deve garantir autenticação (login e senha) para que o acesso à plataforma seja restrito a apenas usuários cadastrados.	Alta	
RF08	O sistema deve permitir que usuários do tipo “Administrador” possam gerenciar o cadastro de	Alta	RF07

Identificador	Descrição	Prioridade	Depende de
	todos os outros usuários e visualizar relatórios consolidados de todas as avaliações por data, nome, sexo dentre outros filtros.		
RF09	O sistema deve permitir que usuários do tipo “Médico” possam apenas realizar o cadastro de seus pacientes, preencher os formulários de triagem e consultar os históricos e relatórios das avaliações realizadas por eles próprios.	Alta	RF07
RF10	O sistema deve notificar o paciente via e-mail, sobre o resultado do exame.	Média	RF05
RF11	O sistema deve permitir que um mesmo paciente faça mais de uma consulta (e possa ser encontrado com base no seu CPF).	Alta	

2.2.4.2. Requisitos Não Funcionais (RNF)

Identificador	Descrição	Prioridade
RNF01	O acesso ao sistema estará disponível na internet, a partir de navegadores disponíveis no mercado.	Alta
RNF02	O sistema deve possuir uma interface intuitiva para que os usuários encontrem as funcionalidades de forma rápida.	Média
RNF03	Os usuários devem ser capazes de utilizar o sistema após verem o vídeo de tutorial.	Média
RNF04	O sistema deve ser implementado com React (Vite) no frontend e Python (FastAPI) no backend.	Alta

Identificador	Descrição	Prioridade
RNF05	O sistema deverá armazenar as informações de acordo com a Lei Geral de Proteção de Dados Pessoais e o Código de Ética Médica.	Alta
RNF06	A interface Web deve ser totalmente responsiva para funcionar em dispositivos móveis (tablets, smartphones) e Desktops.	Alta
RNF07	O resultado do cálculo do score para o encaminhamento ou não ao exame deve ocorrer com tempo inferior ou igual a 3 segundos após o envio do formulário.	Alta

2.3. Deployment (Implantação)

O sistema opera integralmente em ambiente local, sem dependência de serviços de nuvem externos. Todo o stack — frontend, backend e banco de dados — é executado por meio do Docker, utilizando um arquivo docker-compose.yml que orquestra os três serviços em contêineres isolados.

O contêiner do frontend serve a aplicação React compilada via Vite, o contêiner do backend executa a API Python/FastAPI e o contêiner do banco de dados instância o MySQL 8.0 com persistência de dados por volume Docker. Essa abordagem garante que o ambiente seja idêntico para todos os integrantes do grupo, eliminando problemas de dependências do sistema operacional e simplificando a configuração inicial com um único comando.

3. CONCLUSÃO

O sistema desenvolvido para a triagem da Síndrome do X Frágil cumpre o objetivo de transformar um processo clínico complexo em uma ferramenta digital intuitiva e eficiente. Ao automatizar o cálculo do score ponderado e a aplicação dos limiares de decisão (0,56 para homens e 0,55 para mulheres), o software minimiza erros humanos e agiliza o encaminhamento de pacientes para exames confirmatórios, o que é crucial em um sistema de saúde com recursos limitados.

A escolha da arquitetura (React.js, Python/FastAPI e MySQL) e o uso do Docker para implantação local garantem que a plataforma seja não apenas funcional, mas também segura e responsiva, atendendo aos requisitos da LGPD e do Código de Ética Médica. Conclui-se que a digitalização deste checklist clínico fortalece a atuação do Instituto Buko Kaesemodel na disseminação do conhecimento sobre a SXF e na melhoria da qualidade de vida dos pacientes, servindo como uma ponte tecnológica entre a observação clínica e o diagnóstico genético avançado.

REFERÊNCIAS

Fontes sobre Saúde e Síndrome do X Frágil:

REIS, A. C. A Síndrome do X Frágil e seus biomarcadores. 2023. Disponível em: <https://repositorio.usp.br/item/003137891>. Acesso em: 28 abril. 2026.

VARELLA, D. Síndrome do X Frágil. Drauzio Varella, 2024. Disponível em: <https://drauziovarella.uol.com.br/doencas-e-sintomas/sindrome-do-x-fragil/>. Acesso em: 28 abril. 2026.

Documentação Técnica e Ferramentas:

CLOUDFLARE. Cloudflare Pages Documentation. 2024. Disponível em: <https://developers.cloudflare.com/pages/>. Acesso em: 01 mai. 2026.

DOCKER. Docker Documentation. 2024. Disponível em: <https://docs.docker.com/>. Acesso em: 02 mai. 2026.

FASTAPI. FastAPI Documentation. 2024. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 13 mai. 2026.

FIGMA. Figma Help Center. 2024. Disponível em: <https://help.figma.com/hc/en-us>. Acesso em: 25 abril. 2026.

JSPDF. jsPDF Documentation. 2024. Disponível em: <https://rawgit.com/MrRio/jsPDF/master/docs/>. Acesso em: 29 mai. 2026.

MANGUM. Mangum Documentation. 2024. Disponível em:
<https://mangum.fastapiexpert.com/>. Acesso em: 22 mai. 2026.

MYSQL. MySQL 8.0 Reference Manual. 2024. Disponível em:
<https://dev.mysql.com/doc/>. Acesso em: 27 abril. 2026.

NODE.JS. Node.js v20 Documentation. 2024. Disponível em:
<https://nodejs.org/docs/latest/api/>. Acesso em: 27 abril. 2026.

PYTHON. Python 3 Documentation. 2024. Disponível em: <https://docs.python.org/3/>.
Acesso em: 13 mai. 2026.

REACT. React Documentation. 2024. Disponível em: <https://react.dev/>. Acesso em:
25 abril. 2026.

REPORTLAB. ReportLab User Guide. 2024. Disponível em:
<https://www.reportlab.com/docs/reportlab-userguide.pdf>. Acesso em: 25 mai. 2026.

SQLALCHEMY. SQLAlchemy Documentation. 2024. Disponível em:
<https://docs.sqlalchemy.org/>. Acesso em: 16 mai. 2026.

VITE. Vite Guide. 2024. Disponível em: <https://vite.dev/guide/>. Acesso em: 25 abril.
2026.